

Chap.6 합성곱 신경망II (Part 1 ~6.1.4)

6.1 이미지 분류를 위한 신경망

6.1.1 LeNet-5

예제 구현

6.1.2 AlexNet

예제 구현

6.1.3 VGGNet

예제 구현

6.1.4 GoogLeNet

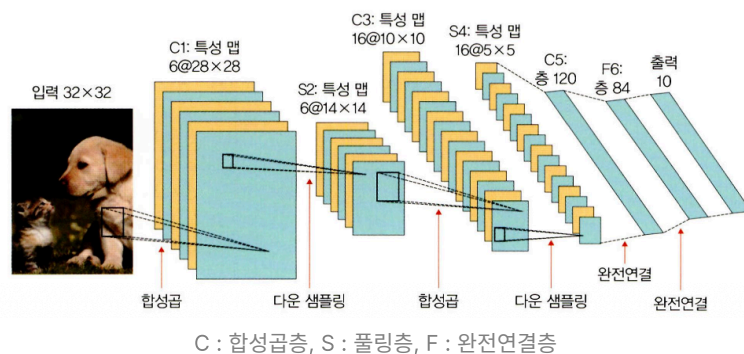
6.1 이미지 분류를 위한 신경망

입력 데이터로 이미지를 사용한 분류(classification)는 특정 대상이 영상 내에 존재하는지 여부를 판단하는 것.

6.1.1 LeNet-5

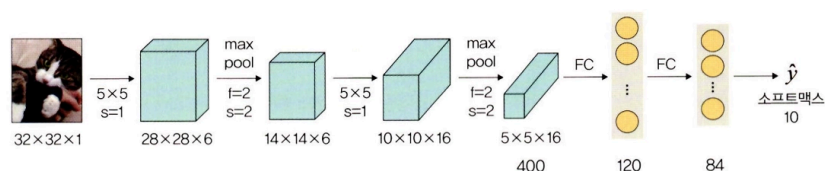
최초로 합성곱 신경망이라는 개념을 개발한 구조 (by. Yann LeCun)

LeNet-5는 합성곱과 다운 샘플링(혹은 풀링)을 반복적으로 거치면서 마지막에 완전연결층에서 분류 수행함.



- C1에서 5x5 합성곱 연산 후 28x28 크기의 특성 맵(feature map) 여섯 개를 생성
- S2에서 다운 샘플링하여 특성 맵 크기를 14x14로 축소
- 다시 C3에서 5x5 합성곱 연산해 10x10 크기의 특성 맵 16개를 생성
- S4에서 다운 샘플링하여 특성 맵 크기를 5x5로 축소
- C5에서 5x5 합성곱 연산하여 1x1 크기의 특성 맵 120개를 생성
- 마지막으로, F6에서 완전 연결층으로 C5의 결과를 유닛(또는 노드) 84개에 연결시킴

예제 구현



32x32 크기의 이미지에 합성곱층과 최대 풀링층이 쌍으로 두 번 적용된 후 완전연결층을 거쳐 이미지가 분류되는 신경망

▼ LeNet-5 예제 신경망 상세

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	32×32	–	–	–
합성곱층	6	28×28	5×5	1	렐루(ReLU)
최대 풀링층	6	14×14	2×2	2	–
합성곱층	16	10×10	5×5	1	렐루(ReLU)
최대 풀링층	16	5×5	2×2	2	–
완전연결층	–	120	–	–	렐루(ReLU)
완전연결층	–	84	–	–	렐루(ReLU)
완전연결층	–	2	–	–	소프트맥스(softmax)

```
# 라이브러리 호출
import torch
import torchvision
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms # 이미지 변환(전처리) 기능 제공하는 라이브러리
from torch.autograd import variable
from torch import optim # 경사하강법 이용해 가중치 구하기 위한 옵티마이저 라이브러리
import torch.nn as nn
import torch.nn.functional as F
import os # 파일 경로에 대한 함수들 제공
import cv2
from PIL import Image
from tqdm import tqdm_notebook as tqdm # 진행 상황을 가시적으로 표현해주는데, 특히 모델의 학습 경과를 확인하고 싶을 때 사용하는 라이브러리
import random
from matplotlib import pyplot as pyplot

device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
print(device)
```

```
# 이미지 데이터셋 전처리
class ImageTransform():
    def __init__(self, resize, mean, std):
        self.data_transform = {
            'train': transforms.Compose([
                transforms.RandomResizedCrop(resize, scale=(0.5, 1.0)),
                transforms.RandomHorizontalFlip(),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ]), # 설명(1)
            'val': transforms.Compose([
                transforms.Resize(256),
```

```

        transforms.CenterCrop(resize),
        transforms.ToTensor(),
        transforms.Normalize(mean, std)
    ])

def __call__(self, img, phase): # 설명(2)
    return self.data_transform[phase](img)

```

- 설명 (1)

`torchvision` 라이브러리를 이용하면 이미지에 대한 전처리를 손쉽게 할 수 있음. `torchvision.transforms` 에서 사용하는 파라미터는 다음과 같다.

```

train_transforms = transforms.Compose([
    transforms.RandomResizedCrop(resize, scale=(0.5, 1.0)),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(mean, std)
])

```

- `transforms.Compose` : 이미지를 변형할 수 있는 방식들의 묶음
- `transforms.RandomResizedCrop` : 입력 이미지를 주어진 크기(resize: 224×224)로 조정. scale은 원래 이미지를 임의의 크기(0.5~1.0(50-100%))만큼 면적을 무작위로 자르겠다는 의미
- `transforms.RandomHorizontalFlip` : 주어진 확률로 이미지를 수평 반전시킴. 이때, 확률 값을 지정하지 않았으므로 기본값인 0.5의 확률로 이미지들이 수평 반전됨. 즉, 훈련 이미지 중 반은 위아래 뒤집힌 상태로 두고, 반은 그대로 사용.
- `transforms.ToTensor` : ImageFolder 메서드를 비롯해 torchvision 메서드는 이미지를 읽을 때 파이썬 이미지 라이브러리인 PIL을 사용. PIL을 사용해 이미지를 읽으면 생성되는 이미지는 범위가 [0, 255]이며, 배열의 차원이 (높이 H x 너비 W x 채널 수 C)로 표현됨. 이후, 효율적 연산을 위해 torch.FloatTensor 배열로 바꾸어야 하는데, 이때 픽셀 값의 범위는 [0.0, 1.0] 사이가 되고 차원의 순서도 (채널 수 C x 높이 H x 너비 W)로 바뀜. 이러한 작업을 수행해 주는 메서드가 ToTensor()
- `transforms.Normalize` : 전이 학습에서 사용하는 사전 훈련된 모델들은 대개 ImageNet 데이터셋에서 훈련 되었음. 따라서 사전 훈련된 모델을 사용하기 위해서는 ImageNet 데이터의 각 채널별 평균과 표준편차에 맞는 정규화를 해주어야 함. 즉, Normalize 메서드 안에 사용된 mean, std는 ImageNet에서 이미지들의 RGB 채널마다 평균과 표준편차를 의미. 참고로 OpenCV를 사용해 이미지 읽어온다면 BGR 이미지(RGB 아님. 채널 순서 주의)

- 설명 (2)

`__call__` 함수는 클래스를 호출할 수 있도록 하는 메서드

`__init__` 은 인스턴스 초기화를 위해 사용한다면, `__call__` 은 인스턴스가 호출되었을 때 실행됨. 즉, 클래스에 `__call__` 함수가 있을 경우, 클래스 객체 자체를 호출하면 `__call__` 함수의 리턴 값이 반환됨

```

# 이미지 데이터셋을 불러온 후 검증, 테스트 분리
cat_directory = r'/Users/eunseoshin/DL-pytorch/chap06/data/dogs-vs-cats/Cat/'
dog_directory = r'/Users/eunseoshin/DL-pytorch/chap06/data/dogs-vs-cats/Dog/'

```

```
g/ '

cat_images_filepaths = sorted([os.path.join(cat_directory, f) for f in os.
    listdir(cat_directory)]) # 설명(1)
dog_images_filepaths = sorted([os.path.join(dog_directory, f) for f in os.
    listdir(dog_directory)])
images_filepaths = [*cat_images_filepaths, *dog_images_filepaths] # 개와 고양이
이미지들을 합쳐서 image_filepaths에 저장
correct_images_filepaths = [i for i in images_filepaths if cv2.imread(i) is
    not None] # 설명(2)

random.seed(42) # 설명(3)
random.shuffle(correct_images_filepaths)
train_images_filepaths = correct_images_filepaths[:400] # 훈련용 400개의 이미지
val_images_filepaths = correct_images_filepaths[400:-10] # 검증용 92개의 이미지
test_images_filepaths = correct_images_filepaths[-10:] # 테스트용 10개의 이미지
print(len(train_images_filepaths), len(val_images_filepaths), len(test_images_filepaths))
```

▼ 코드 실행 결과

400 92 10

• 설명 (1)

고양이 이미지 데이터 가져옴

```
cat_images_filepaths = sorted([os.path.join(cat_directory, f)
                                for f in os.
                                listdir(cat_directory)])
```

- `sorted` : 데이터를 정렬된 리스트로 만들어서 반환
- `os.path.join` : 경로와 파일명을 결합하거나 분할된 경로를 하나로 합치고 싶을 때 사용

▼ c.f. 경로 합치는 방법

```
import os
list_path = ['C:\\', 'Temp', 'user']
folder_path = os.path.join(*list_path)
folder_path

''' 실행결과
'C:\\Temp\\user'
'''
```

- `os.listdir` : 지정한 디렉터리 내 모든 파일의 리스트를 반환함.

• 설명 (2)

```
correct_images_filepaths = [i for i in images_filepaths
                             if cv2.imread
                             (i) is not None]
```

- for 반복문을 이용해 가져온 데이터에 대해 i를 이용해 리스트로 만들
- 반복문을 이용해 images_filepath에서 이미지 데이터를 검색
- 조건문으로 cv2.imread() 함수를 이용해 모든 이미지 데이터를 읽어옴

• 설명 (3)

난수 생성. 시드값 부여해서 나중에 다시 실행해도 동일한 난수 생성하도록

```
# 테스트 데이터셋 이미지 확인 함수
def display_image_grid(images_filepaths, predicted_labels=(), cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows=rows, ncols=cols, figsize=(12,6))
    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # 설명(1)
        true_label = os.path.normpath(image_filepath).split(os.sep)[-2] #
설명(2)
        predicted_label = predicted_labels[i] if predicted_labels else true_label # 설명(3)
        color = "green" if true_label == predicted_label else "red" # 예측
과 정답(레이블)이 동일하면 초록색으로 표시, 그렇지 않다면 빨간색으로 표시
        ax.ravel()[i].imshow(image) # 개별 이미지를 출력
        ax.ravel()[i].set_title(predicted_label, color=color) # predicted_
label을 타이틀로 사용
        ax.ravel()[i].set_axis_off() # 이미지의 축 제거
    plt.tight_layout() # 이미지의 여백을 조정
    plt.show()
```

• 설명 (1)

`cv2.cvtColor` 는 이미지의 색상을 변경하기 위해 사용. 파라미터는 다음과 같다.

```
cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

- `image` : 입력 이미지
- `cv2.COLOR_BGR2RGB` : 변환할 이미지의 색상을 지정하는 것으로 BGR(Blue, Green, Red) 채널 이미지를 RGB(컬러)로 변경하겠다는 의미

• 설명 (2)

이미지의 전체 경로를 정규화하고 분할을 위한 코드. 파라미터는 다음과 같다.

```
true_label = os.path.normpath(image_filepath).split(os.sep)[-2]
```

- `os.path.normpath` : 경로명을 정규화.

- `split(os.sep)` : 경로를 / 또는 \를 기준으로 분할할 때 사용.

- 설명 (3)

```
predicted_label = predicted_labels[i] if predicted_labels else true_label
```

- `predicted_labels` 값이 있으면 그 값을 `predicted_label` 로 사용
- `predicted_labels` 값이 없다면 `true_label` 값을 `predicted_label` 로 사용

```
# 테스트 데이터셋 이미지 출력
display_image_grid(test_images_filepaths)
```

▼ 코드 실행 결과



이제 모델 학습을 위해 데이터셋을 읽어와야 하는데, 크기가 클 수 있으므로 경로만 저장해놓고, `__getitem__` 메서드에서 이미지를 읽어오기. - 배치 크기만큼 분할해 가져옴

```
class DogvsCatDataset(Dataset):
    def __init__(self, file_list, transform=None, phase='train'): # 데이터셋
        # 전처리 (데이터 변형 적용)
        self.file_list = file_list
        self.transform = transform # DogvsCatDataset 클래스를 호출할 때 transform에 대한 매개변수 받아옴
        self.phase = phase # 'train' 적용

    def __len__(self): # images_filepaths 데이터셋의 전체 길이를 반환
        return len(self.file_list)

    def __getitem__(self, idx): # 데이터셋에서 데이터를 가져오는 부분. 결과는 텐서
        # 형태
        img_path = self.file_list[idx]
        img = Image.open(img_path) # img_path 위치에서 이미지 데이터들을 가져옴
        img_transformed = self.transform(img, self.phase) # 이미지에 'train'
        # 전처리를 적용
        label = img_path.split('/')[-1].split('.')[0] # 설명
        if label == 'dog':
            label = 1
```

```
elif label == 'cat':
    label = 0
    return img_transformed, label
```

- 설명

이미지 데이터에 대한 레이블 값(dog, cat)을 가져옴

```
label = img_path.split('/')[-1].split('.')[0]
```

- `img_path` : 이미지가 위치한 전체 경로를 보여줌
- `img_path.split('/')[-1]` : 이미지의 전체 경로에서 '/' 제거
- `split('.')[0]` : 경로에서 마침표 제거 후 첫 번째 값 가져옴

```
# 변수 값 정의
size = 224
mean = (0.485, 0.456, 0.406)
std = (0.229, 0.224, 0.225)
batch_size = 32
```

```
# 이미지 데이터셋 정의
train_dataset = DogvsCatDataset(train_images_filepaths, transform=ImageTransform(size, mean, std), phase='train') # 훈련 이미지에 train_transforms를 적용
val_dataset = DogvsCatDataset(val_images_filepaths, transform=ImageTransform(size, mean, std), phase='val') # 검증 이미지에 test_transforms를 적용

index = 0
print(train_dataset.__getitem__(index)[0].size()) # 훈련 데이터(train_dataset.__getitem__[0][0])의 크기(size()) 출력
print(train_dataset.__getitem__(index)[1]) # 훈련 데이터의 레이블 출력
```

▼ 코드 실행 결과

```
torch.Size([3, 224, 224])
0
```

```
# 데이터로더 정의
train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True) # 설명
val_dataloader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
dataloader_dict = {'train': train_dataloader, 'val': val_dataloader} # 훈련 데이터셋(train_dataloader)과 검증 데이터셋(val_dataloader)을 합쳐서 표현

batch_iterator = iter(train_dataloader)
inputs, label = next(batch_iterator)
```

```
print(inputs.size())
print(label)
```

▼ 코드 실행 결과

```
torch.Size([32, 3, 224, 224])
tensor([1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1,
        0, 0, 0, 0, 0, 0, 1, 1])
```

• 설명

파이토치의 데이터로더는 배치 관리를 담당. 한 번에 모든 데이터를 불러오면 메모리에 부담을 줄 수 있기 때문에 나눠서 조금씩 불러옴

```
train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
```

- 첫 번째 파라미터 : 데이터 불러오기 위한 데이터셋
- `batch_size` : 한 번에 메모리로 불러올 데이터 크기
- `shuffle` : 메모리로 데이터 가져올 때 임의로 섞을건지

```
# 모델의 네트워크 클래스
class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
        self.cnn1 = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=
5, stride=1, padding=0) # 2D 합성곱층이 적용됨. 이때 입력은 (3, 224, 224), 출력
형태는 (weight-kernel_size+1)/stride 에 따라 (16, 220, 220)
        self.relu1 = nn.ReLU()
        self.maxpool1 = nn.MaxPool2d(kernel_size=2) # 적용 후 출력 형태는 22
0/2가 되어 (16, 110, 110)
        self.cnn2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size
=5, stride=1, padding=0) # 출력 형태 (32, 106, 106)
        self.relu2 = nn.ReLU()
        self.maxpool2 = nn.MaxPool2d(kernel_size=2) # 출력 형태 (32, 53, 53)

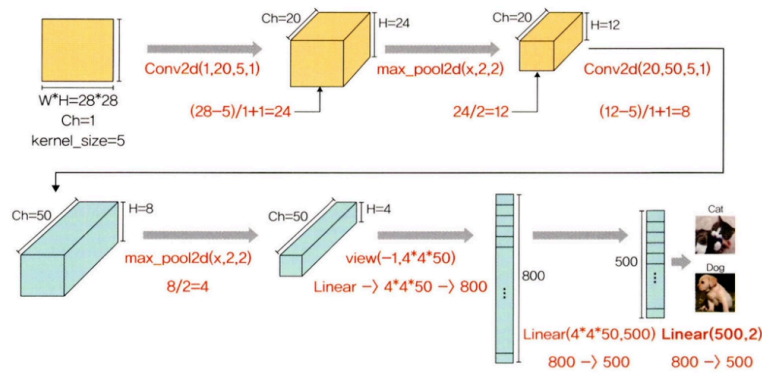
        self.fc1 = nn.Linear(32*53*53, 512)
        self.relu5 = nn.ReLU()
        self.fc2 = nn.Linear(512, 2)
        self.output = nn.Softmax(dim=1)

    def forward(self, x):
        out = self.cnn1(x)
        out = self.relu1(out)
        out = self.maxpool1(out)
        out = self.cnn2(out)
        out = self.relu2(out)
        out = self.maxpool2(out)
        out = out.view(out.size(0), -1) # 완전연결층에 데이터를 전달하기 위해 데
```

이터 형태를 1차원으로 바꿈

```
out = self.fc1(out)
out = self.fc2(out)
out = self.output(out)
return out
```

- 네트워크의 각 부분들을 통과할 때마다 입출력의 형태가 바뀌는데, 그 계산은 아래와 같다.



▼ Conv2d 계층에서 출력 크기 구하기

$$\text{출력 크기} = (W - F + 2P) / S + 1$$

- W : 입력 데이터의 크기(input_volume_size)
- F : 커널 크기(kernel_size)
- P : 패딩 크기(padding_size)
- S : 스트라이드(strides)

▼ MaxPool2d 계층에서의 출력 크기 구하기

$$\text{출력 크기} = IF / F$$

- IF : 입력 필터의 크기(input_filter_size, 또한 바로 앞의 Conv2d의 출력 크기이기도 합니다)
- F : 커널 크기(kernel_size)

```
# 모델 객체 생성
model = LeNet()
print(model)
```

▼ 코드 실행 결과

```

LeNet(
  (cnn1): Conv2d(3, 16, kernel_size=(5, 5), stride=(1, 1))
  (relu1): ReLU()
  (maxpool1): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (cnn2): Conv2d(16, 32, kernel_size=(5, 5), stride=(1, 1))
  (relu2): ReLU()
  (maxpool2): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=89888, out_features=512, bias=True)
  (relu5): ReLU()
  (fc2): Linear(in_features=512, out_features=2, bias=True)
  (output): Softmax(dim=1)
)

```

torchsummary 라이브러리로 출력 결과 확인도 가능

```

# torchsummary 라이브러리 이용한 모델 네트워크 구조 확인
from torchsummary import summary
summary(model, input_size=(3, 224, 224))

```

▼ 코드 실행 결과

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 16, 220, 220]	1,216
ReLU-2	[-1, 16, 220, 220]	0
MaxPool2d-3	[-1, 16, 110, 110]	0
Conv2d-4	[-1, 32, 106, 106]	12,832
ReLU-5	[-1, 32, 106, 106]	0
MaxPool2d-6	[-1, 32, 53, 53]	0
Linear-7	[-1, 512]	46,023,168
Linear-8	[-1, 2]	1,026
Softmax-9	[-1, 2]	0

Total params: 46,038,242
 Trainable params: 46,038,242
 Non-trainable params: 0

Input size (MB): 0.57
 Forward/backward pass size (MB): 19.47
 Params size (MB): 175.62
 Estimated Total Size (MB): 195.67

- summary를 통해 모델 네트워크 관련 정보 확인 가능

```
summary(model, input_size=(3, 224, 224))
```

- 첫 번째 파라미터 : 모델의 네트워크
- 두 번째 파라미터 : 입력 값 (channel, width, height)

```

# 학습 가능한 파라미터 수 확인
def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

print(f'The model has {count_parameters(model):,} trainable parameters')

```

▼ 코드 실행 결과

The model has 46,038,242 trainable parameters

```
# 옵티마이저와 손실 함수 정의
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
criterion = nn.CrossEntropyLoss()
```

- optimizer

경사하강법으로 모멘텀 SGD를 사용하도록 함. 모멘텀 SGD는 관성이 추가된 것으로 매번 기울기를 구하지만 가중치를 수정하기 전에 이전 수정 방향(+, -)을 참고하여 같은 방향으로 일정한 비율만 수정되게 하는 방법

```
optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
```

- 첫 번째 파라미터 : 경사 하강법을 통해 궁극적으로 업데이트 하고자 하는 파라미터는 weight와 bias.
- `lr(learning rate)` : 가중치를 변경할 때 얼마나 크게 변경할지 결정
- `momentum` : SGD를 적절한 방향으로 가속화하며, 진동을 줄여주는 매개변수

```
# 모델의 파라미터와 손실 함수를 device(mps)에 할당
model = model.to(device)
criterion = criterion.to(device)
```

```
# 모델 학습 함수 정의
def train_model(model, dataloader_dict, criterion, optimizer, num_epoch):
    since = time.time()
    best_acc = 0.0

    for epoch in range(num_epoch): # epoch를 10이기 때문에 10회 반복
        print('Epoch {}/{}'.format(epoch+1, num_epoch))
        print('-'*20)

        for phase in ['train', 'val']:
            if phase == 'train':
                model.train() # 모델 학습
            else:
                model.eval()

            epoch_loss = 0.0
            epoch_corrects = 0

            for inputs, labels in tqdm(dataloader_dict[phase]): # dataloader_dict는 train_loader를 의미
                inputs = inputs.to(device)
                labels = labels.to(device)
                optimizer.zero_grad()

                with torch.set_grad_enabled(phase == 'train'):
                    outputs = model(inputs)
                    _, preds = torch.max(outputs, 1)
```

```

        loss = criterion(outputs, labels) # 손실 함수를 이용한 오
차 계산

        if phase == 'train':
            loss.backward() # 모델의 학습 가능한 모든 파라미터에 대
해 기울기 계산
            optimizer.step() # optimizer의 step 함수 호출하면 파라
미터 갱신

            epoch_loss += loss.item() * inputs.size(0) # 설명
            epoch_corrects += torch.sum(preds == labels.data) # 정
답과 예측이 일치하면 그것의 합계를 epoch_corrects에 저장

            epoch_loss = epoch_loss / len(dataloader_dict[phase].datas
et) # 최종 오차 계산 (오차를 데이터셋 길이(개수)로 나눠서 계산)
            epoch_acc = epoch_corrects.double() / len(dataloader_dict
[phase].dataset) # 최종 정확도(epoch_corrects를 데이터셋의 길이로 나눠서 계산)

            print('{} Loss: {:.4f} Acc: {:.4f}'.format(phase, epoch_lo
ss, epoch_acc))

            if phase == 'val' and epoch_acc > best_acc: # 검증 데이터셋에
대한 가장 최적의 정확도 저장
                best_acc = epoch_acc
                best_model_wts = model.state_dict()

            time_elapsed = time.time() - since
            print('Training complete in {:.0f}m {:.0f}s'.format(time_elapsed
// 60, time_elapsed % 60))
            print('Best val Acc: {:.4f}'.format(best_acc))
            return model

```

- 설명

오차와 입력을 곱하는 이유? → 손실 함수의 reduction이라는 파라미터 알아보기

손실함수 정의할 때, `criterion = nn.CrossEntropyLoss(reduction='mean')` 이렇게 정의할 수 있음. (앞에서는 기
본값 사용해서 명시 X)

default 값은 'mean'. 정답과 예측 값의 오차를 구한 후 그 값들의 평균 반환

```

# 모델 학습
import time

num_epoch = 10
model = train_model(model, dataloader_dict, criterion, optimizer, num_epoc
h)

```

▼ 코드 실행 결과

```

Epoch 1/10
-----
/var/folders/4q/3301trc50fz6_vl0qx1b71r00000gn/T/ipykernel_1891/3160953479.py:
Please use `tqdm.notebook.tqdm` instead of `tqdm.tqdm_notebook`
for inputs, labels in tqdm(dataloader_dict[phase]): # dataloader_dict= train
100%  13/13 [00:01<00:00, 6.90it/s]
train Loss: 0.6890 Acc: 0.5400
100%  3/3 [00:00<00:00, 11.04it/s]
val Loss: 0.6927 Acc: 0.5326
Epoch 2/10
-----
100%  13/13 [00:01<00:00, 7.36it/s]
train Loss: 0.6861 Acc: 0.5275
100%  3/3 [00:00<00:00, 12.30it/s]
val Loss: 0.6942 Acc: 0.5000
Epoch 3/10
-----
100%  13/13 [00:01<00:00, 7.43it/s]
train Loss: 0.6793 Acc: 0.5775
100%  3/3 [00:00<00:00, 12.51it/s]
val Loss: 0.6937 Acc: 0.5761
Epoch 4/10
-----
100%  13/13 [00:01<00:00, 7.27it/s]
train Loss: 0.6831 Acc: 0.5575
100%  3/3 [00:00<00:00, 12.17it/s]
val Loss: 0.6946 Acc: 0.5543
Epoch 5/10
-----
100%  13/13 [00:01<00:00, 7.17it/s]
train Loss: 0.6792 Acc: 0.5850
100%  3/3 [00:00<00:00, 11.86it/s]
val Loss: 0.6853 Acc: 0.6196
Epoch 6/10
-----
100%  13/13 [00:01<00:00, 7.26it/s]
train Loss: 0.6685 Acc: 0.5950
100%  3/3 [00:00<00:00, 11.68it/s]
val Loss: 0.6847 Acc: 0.5978
Epoch 7/10
-----
100%  13/13 [00:01<00:00, 7.43it/s]
train Loss: 0.6618 Acc: 0.6300
100%  3/3 [00:00<00:00, 11.28it/s]
val Loss: 0.6767 Acc: 0.5435
Epoch 8/10
-----
100%  13/13 [00:01<00:00, 7.08it/s]
train Loss: 0.6548 Acc: 0.6475
100%  3/3 [00:00<00:00, 12.41it/s]
val Loss: 0.6756 Acc: 0.5870

```

```

Epoch 9/10
-----
100% ██████████ 13/13 [00:01<00:00, 6.80it/s]
train Loss: 0.6446 Acc: 0.6525
100% ██████████ 3/3 [00:00<00:00, 11.54it/s]
val Loss: 0.6732 Acc: 0.5652
Epoch 10/10
-----
100% ██████████ 13/13 [00:01<00:00, 5.72it/s]
train Loss: 0.6442 Acc: 0.6225
100% ██████████ 3/3 [00:00<00:00, 9.60it/s]
val Loss: 0.6739 Acc: 0.5543
Training complete in 0m 21s
Best val Acc: 0.6196

```

검증 데이터셋을 이용한 모델 학습 결과, 최고 62%의 정확도를 보임. → 높지 않은 결과
 좀 더 정확한 결과를 위해선 데이터셋을 늘려야 함.

```

# 모델 테스트를 위한 함수 정의
import pandas as pd

id_list = []
pred_list = []
_id = 0
with torch.no_grad(): # 역전파 중 텐서들에 대한 변화도를 계산할 필요가 없음을 나타냄.
    # 훈련 데이터셋의 모델 학습과 가장 큰 차이점
    for test_path in tqdm(test_images_filepaths): # 테스트 데이터셋 이용
        img = Image.open(test_path)
        _id = test_path.split('/')[-1].split('.')[1]
        transform = ImageTransform(size, mean, std)
        img = transform(img, phase='val') # 테스트 데이터셋 전처리 적용
        img = img.unsqueeze(0) # 설명(1)
        img = img.to(device)

        model.eval()
        outputs = model(img)
        preds = F.softmax(outputs, dim=1)[: , 1].tolist() # 설명(2)
        id_list.append(_id)
        pred_list.append(preds[0])

res = pd.DataFrame({
    'id': id_list,
    'label': pred_list
}) # 테스트 데이터셋의 예측 결과인 id와 레이블(label)을 데이터 프레임에 저장

res.sort_values(by='id', inplace=True)
res.reset_index(drop=True, inplace=True)

res.to_csv('../chap06/data/LeNet', index=False) # 데이터 프레임을 CSV 파일로 저장

```

▼ 코드 실행 결과 (처리 완료)

```
/var/folders/4g/3301trc50fz6_vl0gx1b71r00000gn/T/ipykernel_1891/4077494656.py:8: `
Please use `tqdm.notebook.tqdm` instead of `tqdm.tqdm_notebook`
  for test_path in tqdm(test_images_filepaths): # 테스트 데이터셋 이용
100% ██████████ 10/10 [00:00<00:00, 4.80it/s]
```

• 설명 (1)

`torch.unsqueeze` 는 텐서에 차원을 추가할 때 사용. (0)은 차원이 추가될 위치를 의미함.

• 설명 (2)

softmax는 지정된 차원(dim)을 따라 텐서의 요소(텐서의 개별 값)가 (0, 1) 범위에 있고 합계가 1이 되도록 크기 다시 조정

```
F.softmax(outputs, dim=1)[:, 1].tolist()
```

- `outputs, dim=1` : outputs에 softmax를 적용해 각 행의 합이 1이 되도록 함
- `[:, 1]` : (outputs, dim=1)의 값 중 모든 행(:)에서 두 번째 칼럼(1번째 인덱스)을 가져옴
- `tolist()` : 배열을 리스트 형태로 변환

```
# 테스트 데이터셋의 예측 결과 호출
res.head(10)
```

▼ 코드 실행 결과

	id	label
0	109	0.507585
1	145	0.408364
2	15	0.649367
3	162	0.495898
4	167	0.494895
5	200	0.516006
6	210	0.676380
7	211	0.580432
8	213	0.371393
9	224	0.611713

label이 0.5보다 크면 개, 작으면 고양이를 의미

```
# 테스트 데이터셋 이미지를 출력하기 위한 함수 정의
class_ = classes = {0:'cat', 1:'dog'}
def display_image_grid(images_filepaths, predicted_labels=(), cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows=rows, ncols=cols, figsize=(12, 6))
    for i, images_filepath in enumerate(images_filepaths):
        image = cv2.imread(images_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        a = random.choice(res['id'].values) # 데이터 프레임의 id라는 칼럼에서
```

임의로 데이터를 가져옴

```
label = res.loc[res['id'] == a, 'label'].values[0]
if label > 0.5: # 0.5보다 크면 개
    label = 1
else: # 0.5보다 작으면 고양이
    label = 0
ax.ravel()[i].imshow(image)
ax.ravel()[i].set_title(class_[label])
ax.ravel()[i].set_axis_off()
```

```
plt.tight_layout()
plt.show()
```

```
# 테스트 데이터셋 예측 결과 이미지 출력
display_image_grid(test_images_filepaths)
```

▼ 코드 실행 결과

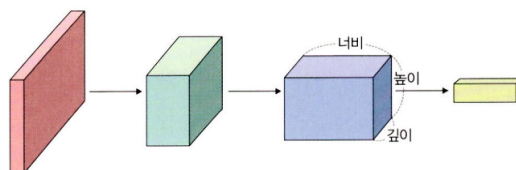


예측력이 좋지는 않음. 극히 일부의 데이터를 이용한 모델 학습을 진행했기 때문.

6.1.2 AlexNet

AlexNet은 ImageNet 영상 데이터베이스를 기반으로 한 화상 인식 대회인 'ILSVRC 2012'에서 우승한 CNN 구조.

c.f. CNN 구조

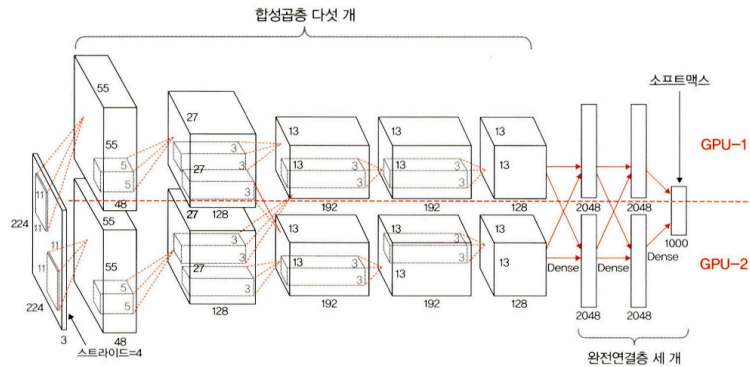


이미지를 다루기 때문에 기본적으로 3차원 데이터를 다룸.

이미지 크기를 나타내는 너비(width), 높이(height) + 깊이(depth)를 가짐.

보통 색상이 많은 이미지는 R/G/B 성분 세 개를 갖기 때문에 시작이 3이지만, 합성곱을 거치며 특성 맵이 만들어지고 이것에 따라 중간 영상의 깊이가 달라짐.

AlexNet은 합성곱층 총 5개와 완전연결층 3개로 구성되어 있으며, 맨 마지막 완전연결층은 카테고리 1000개를 분류하기 위해 소프트맥스 활성화 함수 사용. 전체적으로 보면 GPU 두 개를 기반으로 한 병렬 구조인 점을 제외하면 LeNet-5와 크게 다르지 않음.



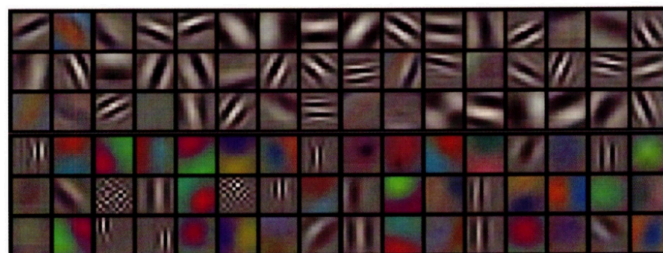
AlexNet의 합성곱층에서 사용된 활성화 함수는 ReLU로, 각 계층의 구조적 세부사항은 다음 표 참고.

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	227×227	—	—	—
합성곱층	96	55×55	11×11	4	렐루(ReLU)
최대 풀링층	96	27×27	3×3	2	—
합성곱층	256	27×27	5×5	1	렐루(ReLU)
최대 풀링층	256	13×13	3×3	2	—
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	256	13×13	3×3	1	렐루(ReLU)
최대 풀링층	256	6×6	3×3	2	—
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	1000	—	—	소프트맥스(softmax)

네트워크에는 학습 가능한 변수가 총 6600만 개가 있음. 네트워크에 대한 입력은 227 x 227 x 3 크기의 RGB 이미지이며, 각 클래스(혹은 카테고리)에 해당하는 1000 x 1 확률 벡터를 출력.

AlexNet의 첫 번째 합성곱층 커널의 크기는 11 x 11 x 3 이며, stride를 4로 적용하여 특성 맵을 96개 생성하기 때문에 55 x 55 x 96의 출력을 가짐. 첫 번째 계층을 거치며 GPU-1에서는 주로 컬러와 상관 없는 정보를 추출하기 위한 커널이 학습되고, GPU-2에서는 주로 컬러와 관련된 정보를 추출하기 위한 커널이 학습됨.

▼ 그림 6-10 AlexNet GPU-1 · 2 적용 결과



예제 구현

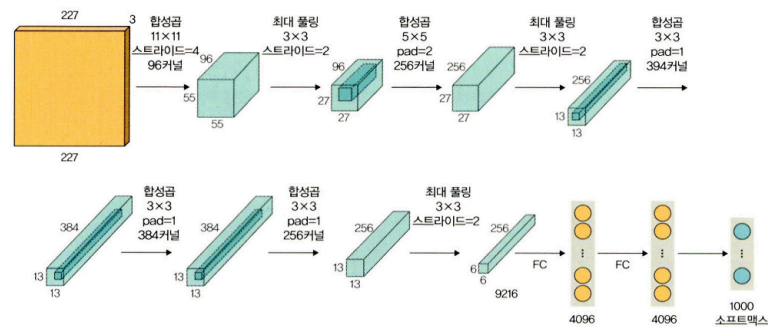
전반적인 코드는 LeNet과 크게 다르지 않기 때문에, 주로 모델을 구성하는 네트워크의 차이 위주로 살펴보기

```
# 필요한 라이브러리 호출
import torch
import torchvision
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms
from torch.autograd import Variable
from torch import optim
import torch.nn as nn
import torch.nn.functional as F
import os
import cv2
import random
from PIL import Image
from tqdm import tqdm_notebook as tqdm

device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
print(device)
```

원래 AlexNet의 마지막 계층에는 뉴런이 1000개 있지만, 예제에서는 클래스 두 개만 사용

▼ AlexNet 예제 네트워크



```
# 이미지 데이터셋 전처리
class ImageTransform():
    def __init__(self, resize, mean, std):
        self.data_transform = {
            'train': transforms.Compose([
                transforms.RandomResizedCrop(resize, scale=(0.5, 1.0)),
                transforms.RandomHorizontalFlip(),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ]),
```

```

        'val': transforms.Compose([
            transforms.Resize(256),
            transforms.CenterCrop(resize),
            transforms.ToTensor(),
            transforms.Normalize(mean, std)
        ])
    }

    def __call__(self, img, phase):
        return self.data_transform[phase](img)

```

```

# 이미지 데이터셋을 불러온 후 검증, 테스트 분리
cat_directory = r'/Users/eunseoshin/DL-pytorch/chap06/data/dogs-vs-cats/Cat/'
dog_directory = r'/Users/eunseoshin/DL-pytorch/chap06/data/dogs-vs-cats/Dog/'

cat_images_filepaths = sorted([os.path.join(cat_directory, f) for f in os.
    listdir(cat_directory)])
dog_images_filepaths = sorted([os.path.join(dog_directory, f) for f in os.
    listdir(dog_directory)])
images_filepaths = [*cat_images_filepaths, *dog_images_filepaths] # 개와 고양이 이미지들을 합쳐서 image_filepaths에 저장
correct_images_filepaths = [i for i in images_filepaths if cv2.imread(i) is not None]

random.seed(42)
random.shuffle(correct_images_filepaths)
train_images_filepaths = correct_images_filepaths[:400] # 훈련용 400개의 이미지
val_images_filepaths = correct_images_filepaths[400:-10] # 검증용 92개의 이미지
test_images_filepaths = correct_images_filepaths[-10:] # 테스트용 열 개의 이미지
print(len(train_images_filepaths), len(val_images_filepaths), len(test_images_filepaths))

```

▼ 코드 실행 결과

400 92 10

사용하는 노트북 성능이 좋지 않다고 가정하고 진행 → 훈련 데이터셋 400개로 제한

AlexNet은 파라미터를 6000만 개 사용하는 모델. 충분한 데이터가 없으면 과적합이 발생하는 등 테스트 데이터에 대한 성능이 좋지 않음. 성능 좋은 결과를 원한다면 충분한 데이터셋을 확보해서 테스트 진행하기.

```

# 커스텀 데이터셋 정의
class DogvsCatDataset(Dataset):
    def __init__(self, file_list, transform=None, phase='train'):

```

```

self.file_list = file_list # 이미지 데이터가 위치한 파일 경로
self.transform = transform # 이미지 데이터 전처리
self.phase = phase # self.phase는 ImageTransform()에서 정의한 'train'과 'val'을 의미

def __len__(self): # images_filepaths 데이터셋의 전체 길이를 반환
    return len(self.file_list)

def __getitem__(self, idx): # 데이터셋에서 데이터를 가져오는 부분. 결과는 텐서 형태
    img_path = self.file_list[idx] # 이미지 데이터의 인덱스 가져오기
    img = Image.open(img_path) # img_path 위치에서 이미지 데이터들을 가져옴
    img_transformed = self.transform(img, self.phase) # 이미지에 'train' 전처리를 적용

    label = img_path.split('/')[-1].split('.')[0] # 레이블 값 가져오기
    if label == 'dog':
        label = 1
    elif label == 'cat':
        label = 0

    return img_transformed, label # 전처리가 적용된 이미지와 레이블 반환

```

```

# 변수에 대한 값 정의
size = 256 # AlexNet은 깊이가 깊은 네트워크를 사용하므로 이미지 크기가 256이 아니면
풀링층 때문에 크기가 계속 줄어들어 오류가 발생할 수 있음
mean = (0.485, 0.456, 0.406)
std = (0.229, 0.224, 0.225)
batch_size = 32

```

```

# 훈련, 검증, 테스트 데이터셋 정의
train_dataset = DogvsCatDataset(train_images_filepaths, transform=ImageTransform(size, mean, std), phase='train') # 훈련 이미지에 train_transforms를 적용
val_dataset = DogvsCatDataset(val_images_filepaths, transform=ImageTransform(size, mean, std), phase='val') # 검증 이미지에 test_transforms를 적용

index = 0
print(train_dataset.__getitem__(index)[0].size()) # 훈련 데이터(train_dataset.__getitem__[0][0])의 크기(size()) 출력
print(train_dataset.__getitem__(index)[1]) # 훈련 데이터의 레이블 출력

```

▼ 코드 실행 결과

```

torch.Size([3, 256, 256])
0

```

```
# 데이터셋을 메모리로 불러옴
train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_dataloader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
dataloader_dict = {'train': train_dataloader, 'val': val_dataloader} # 훈련
데이터셋(train_dataloader)과 검증 데이터셋(val_dataloader)을 합쳐서 표현

batch_iterator = iter(train_dataloader)
inputs, label = next(batch_iterator)
print(inputs.size())
print(label)
```

▼ 코드 실행 결과

```
torch.Size([32, 3, 256, 256])
tensor([0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0])
```

AlexNet 모델의 네트워크 - Conv2d + ReLU + MaxPool2d 가 다섯 번 반복된 후 두 개의 완전연결층과 출력층으로 구성된 네트워크

```
# AlexNet 모델 네트워크 정의
class AlexNet(nn.Module):
    def __init__(self) -> None:
        super(AlexNet, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=11, stride=4, padding=2),
            nn.ReLU(inplace=True), # 설명(1)
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(64, 128, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2),
            nn.Conv2d(128, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(256, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2)
        )
        self.avgpool = nn.AdaptiveAvgPool2d((6, 6)) # 설명(2)
        self.classifier = nn.Sequential(
            nn.Dropout(),
            nn.Linear(256*6*6, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(),
            nn.Linear(4096, 512),
            nn.ReLU(inplace=True),
            nn.Linear(512, 2)
```

```

    )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.features(x)
        x = self.avgpool(x)
        x = torch.flatten(x, 1)
        x = self.classifier(x)
        return x

```

- 설명 (1)

inplace 연산이란 연산에 대한 결과값을 새로운 변수에 저장하는 것이 아닌 기존 데이터를 대체하는 것을 의미함. 즉, 기존 값을 연산 결과값으로 대체함으로써 기존 값들을 무시하겠다는 의미.

- 설명 (2)

`nn.AdaptiveAvgPool2d` 는 `nn.AvgPool2d` 처럼 풀링을 위해 사용함.

- AvgPool2d에서는 풀링에 대한 커널 및 스트라이드 크기를 정의해야 동작함.

e.g. `nn.AvgPool2d((3, 2), stride=(2, 1))`처럼 지정하면 그 결과는 5 x 5 텐서를 3 x 3 텐서로, 7 x 7 텐서를 4 x 4 텐서로 줄이는 효과를 얻을 수 있음.

(N, C, H_{in}, W_{in}) 크기의 입력을 (N, C, H_{out}, W_{out}) 크기로 출력하는 것이 AvgPool2d

$$H_{out} = \left\lceil \frac{H_{in} + 2 \times \text{padding}[0] - \text{kernel_size}[0]}{\text{stride}[0]} + 1 \right\rceil$$

$$W_{out} = \left\lceil \frac{W_{in} + 2 \times \text{padding}[1] - \text{kernel_size}[1]}{\text{stride}[1]} + 1 \right\rceil$$

- AdaptiveAvgPool2d는 풀링 작업이 끝날 때 필요한 출력 크기를 정의함.

`nn.AvgPool2d`에서는 커널 크기, 스트라이드, 패딩을 지정했다면, `nn.AdaptiveAvgPool2d`는 출력에 대한 크기만 지정함. 따라서 AdaptiveAvgPool2d를 사용할 경우 출력 크기에 대한 조정이 상당히 쉬워짐.

참고로 AdaptiveAvgPool2d는 입력 크기에 변동이 있고 CNN 위쪽에 완전연결층을 사용하는 경우 유용함.

```

# model 객체 생성
model = AlexNet()
model.to(device)

```

▼ 코드 실행 결과

```

AlexNet(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(11, 11), stride=(4, 4), padding=(2, 2))
    (1): ReLU(inplace=True)
    (2): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    (3): Conv2d(64, 192, kernel_size=(5, 5), stride=(1, 1), padding=(2, 2))
    (4): ReLU(inplace=True)
    (5): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
    (6): Conv2d(192, 384, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (7): ReLU(inplace=True)
    (8): Conv2d(384, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): ReLU(inplace=True)
    (10): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (11): ReLU(inplace=True)
    (12): MaxPool2d(kernel_size=3, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(6, 6))
  (classifier): Sequential(
    (0): Dropout(p=0.5, inplace=False)
    (1): Linear(in_features=9216, out_features=4096, bias=True)
    (2): ReLU(inplace=True)
    (3): Dropout(p=0.5, inplace=False)
    (4): Linear(in_features=4096, out_features=512, bias=True)
    (5): ReLU(inplace=True)
    (6): Linear(in_features=512, out_features=2, bias=True)
  )
)

```

옵티마이저 및 손실 함수 정의

```

optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
criterion = nn.CrossEntropyLoss()

```

모델 학습 함수 정의

```

def train_model(model, dataloader_dict, criterion, optimizer, num_epoch):
    since = time.time()
    best_acc = 0.0

    for epoch in range(num_epoch):
        print('Epoch {}/{}'.format(epoch+1, num_epoch))
        print('-'*20)

        for phase in ['train', 'val']:
            if phase == 'train':
                model.train()
            else:
                model.eval()

        epoch_loss = 0.0
        epoch_corrects = 0

        for inputs, labels in tqdm(dataloader_dict[phase]):
            inputs = inputs.to(device)
            labels = labels.to(device)
            optimizer.zero_grad()

```

```

        with torch.set_grad_enabled(phase == 'train'):
            outputs = model(inputs)
            _, preds = torch.max(outputs, 1)
            loss = criterion(outputs, labels)

            if phase == 'train':
                loss.backward()
                optimizer.step()

            epoch_loss += loss.item() * inputs.size(0)
            epoch_corrects += torch.sum(preds == labels.data)

    epoch_loss = epoch_loss / len(dataloader_dict[phase].dataset)
    epoch_acc = epoch_corrects.float() / len(dataloader_dict[phase].dataset)

    print('{} Loss: {:.4f} Acc: {:.4f}'.format(phase, epoch_loss, epoch_acc))

    time_elapsed = time.time() - since
    print('Training complete in {:.0f}m {:.0f}s'.format(time_elapsed // 60, time_elapsed % 60))
    print('Best val Acc: {:.4f}'.format(best_acc))
    return model

```

```

# 모델 학습
import time

num_epoch = 10
model = train_model(model, dataloader_dict, criterion, optimizer, num_epoch)

```

▼ 코드 실행 결과

```

Epoch 1/10
-----
/var/folders/4g/3301trc50fz6_vl0gx1b71r00000qn/T/ipykernel_13086/1315517893.py:19:
Please use `tqdm.notebook.tqdm` instead of `tqdm.tqdm_notebook`
  for inputs, labels in tqdm(dataloader_dict[phase]):
100% ██████████ 13/13 [00:07<00:00, 2.07it/s]
train Loss: 0.6933 Acc: 0.4975
100% ██████████ 3/3 [00:00<00:00, 3.50it/s]
val Loss: 0.6935 Acc: 0.4891
Epoch 2/10
-----
100% ██████████ 13/13 [00:06<00:00, 2.08it/s]
train Loss: 0.6932 Acc: 0.4950
100% ██████████ 3/3 [00:00<00:00, 3.40it/s]
val Loss: 0.6933 Acc: 0.4891

```

... (생략)

```
Epoch 10/10
-----
100% ██████████ 13/13 [00:07<00:00, 1.91it/s]
train Loss: 0.6930 Acc: 0.5175
100% ██████████ 3/3 [00:00<00:00, 3.43it/s]
val Loss: 0.6929 Acc: 0.5435
Training complete in 1m 23s
```

```
# 모델을 이용한 예측
import pandas as pd

id_list = []
pred_list = []
_id = 0
with torch.no_grad():
    for test_path in tqdm(test_images_filepaths): # 테스트 데이터셋 이용
        img = Image.open(test_path)
        _id = test_path.split('/')[1].split('.')[1] # 이미지 데이터의 번호 가
저오기
        transform = ImageTransform(size, mean, std)
        img = transform(img, phase='val') # 테스트 데이터셋 전처리 적용
        img = img.unsqueeze(0)
        img = img.to(device)

        model.eval()
        outputs = model(img)
        preds = F.softmax(outputs, dim=1)[:1].tolist()

        id_list.append(_id)
        pred_list.append(preds[0])

res = pd.DataFrame({
    'id': id_list,
    'label': pred_list
}) # 데이터 프레임에 이미지의 id(번호)와 레이블 저장

res.to_csv('../chap06/data/LeNet', index=False) # 데이터 프레임을 CSV 파일로
저장
```

▼ 코드 실행 결과

```
/var/folders/4g/3301trc50fz6\_vl0qx1b71r00000gn/T/ipykernel\_13006/1412150534.py:8:
Please use `tqdm.notebook.tqdm` instead of `tqdm.tqdm_notebook`
    for test_path in tqdm(test_images_filepaths): # 테스트 데이터셋 이용
100% ██████████ 10/10 [00:00<00:00, 45.48it/s]
```

```
# 데이터 프레임 결과 확인
res.head(10)
```

▼ 코드 실행 결과

	id	label
0	145	0.499688
1	211	0.500371
2	162	0.499763
3	200	0.500864
4	210	0.499757
5	224	0.499503
6	213	0.499510
7	109	0.500507
8	15	0.499651
9	167	0.499509

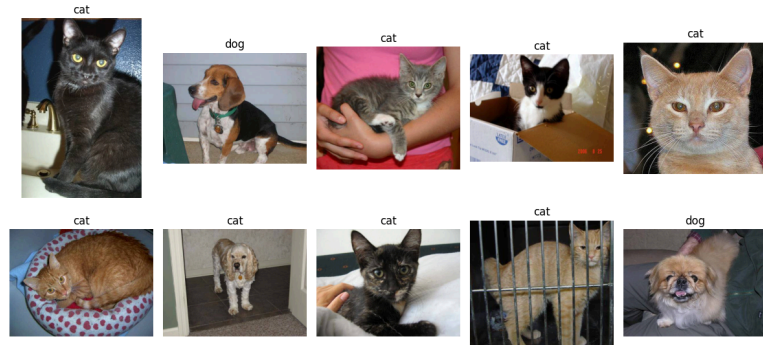
```
# 예측 결과를 시각적으로 표현하기 위한 함수 정의
class_ = classes = {0:'cat', 1:'dog'}
def display_image_grid(images_filepaths, predicted_labels=(), cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows=rows, ncols=cols, figsize=(12, 6))
    for i, images_filepath in enumerate(images_filepaths):
        image = cv2.imread(images_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        a = random.choice(res['id'].values)
        label = res.loc[res['id'] == a, 'label'].values[0]
        if label > 0.5:
            label = 1
        else:
            label = 0
        ax.ravel()[i].imshow(image)
        ax.ravel()[i].set_title(class_[label])
        ax.ravel()[i].set_axis_off()

    plt.tight_layout()
    plt.show()
```

```
# 예측 결과에 대해 이미지와 함께 출력
from matplotlib import pyplot as plt
display_image_grid(test_images_filepaths)
```

▼ 코드 실행 결과



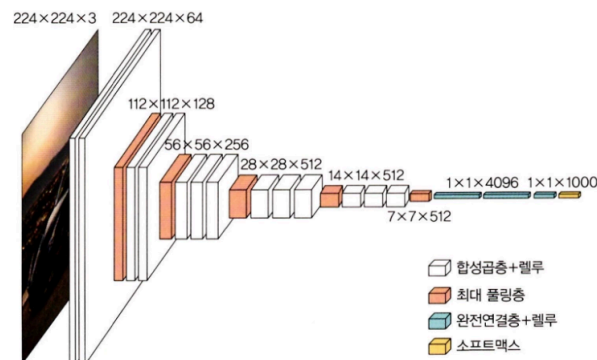
6.1.3 VGGNet

2015 ICLR에 게재된 “Very deep convolutional networks for large-scale image recognition” 논문에서 처음 발표함.

VGGNet은 합성곱층의 파라미터 수를 줄이고 훈련 시간을 개선하려고 탄생. 즉, 네트워크를 깊게 만드는 것이 성능에 어떤 영향을 미치는지 확인하고자 나온 것.

VGG 연구팀은 깊이의 영향만 최대한 확인하고자 합성곱층에서 사용하는 필터/커널의 크기를 가장 작은 3x3으로 고정함.

네트워크 계층의 총 개수에 따라 여러 유형의 VGGNet(VGG16, VGG19 등)이 있으며, 이 중 VGG16 네트워크의 구조적 세부 사항은 다음 그림과 같다.



계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	224×224	—	—	—
합성곱층	64	224×224	3×3	1	렐루(ReLU)
합성곱층	64	224×224	3×3	1	렐루(ReLU)
최대 풀링층	64	112×112	2×2	2	—
합성곱층	128	112×112	3×3	1	렐루(ReLU)
합성곱층	128	112×112	3×3	1	렐루(ReLU)
최대 풀링층	128	56×56	2×2	2	—
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
최대 풀링층	256	28×28	2×2	2	—
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
최대 풀링층	512	14×14	2×2	2	—

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
최대 풀링층	512	7×7	2×2	2	-
완전연결층	-	4096	-	-	렐루(ReLU)
완전연결층	-	4096	-	-	렐루(ReLU)
완전연결층	-	1000	-	-	소프트맥스(softmax)

VGG16에는 파라미터가 총 1억 3300만 개 있음. 여기에서 주목할 점은 모든 합성곱 커널의 크기는 3x3, 최대 풀링 커널의 크기는 2x2이며, 스트라이드는 2라는 것. 결과적으로 64개의 224x224 특성 맵(224x224x64)들이 생성됨. 또한, 마지막 16번째 계층을 제외하고는 모두 ReLU 활성화 함수가 적용됨.

예제 구현

VGGNet 중 가장 간단한 VGG11을 파이토치로 구현.

VGG16이나 VGG19를 구현해보고 싶다면, VGG11에서 더 깊게 쌓아 올리면 됨.

```
# 필요한 라이브러리 호출
import copy # 설명
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torch.utils.data as data
import torchvision
from torchvision import transforms
import torchvision.datasets as Datasets

device = torch.device("cpu")
```

- 설명

객체 복사를 위해 사용. 객체 복사는 크게 얇은 복사(shallow copy)와 깊은 복사(deep copy)로 나뉨.

- 단순한 객체 복사

```
original = [1, 2, 3] # original이라는 변수에 [1, 2, 3] 저장
copy_o = original # copy_o에 original 복사(shallow copy)
print(copy_o)

copy_o[2] = 10 # copy_o의 3을 10으로 값을 변경
print(copy_o)
print(original)

''' 결과
[1, 2, 3]
[1, 2, 10]
```

```
[1, 2, 10]
...
```

copy_o 뿐만 아니라 원래 값인 original의 3도 10으로 바뀜

- 얕은 복사. copy.copy() 사용

```
import copy

original = [[1, 2], 3]
copy_o = copy.copy(original) ----- original 값을 copy_o에 얕은 복사(copy.copy())
print(copy_o)
copy_o[0] = 100 ----- copy_o의 [1, 2] 값을 100으로 변경
print(copy_o)
print(original)

append = copy.copy(original)
append[0].append(4) ----- 첫 번째 리스트([1, 2])에 4를 추가
print(append)
print(original)
```

결과

```
[[1, 2], 3]
[100, 3]
[[1, 2], 3]
[[1, 2, 4], 3]
[[1, 2, 4], 3]
```

100으로 변경한 것 → copy_o만 바뀜.

4 추가 → original과 copy_o 모두 반영

- 깊은 복사. copy.deepcopy() 이용

```
import copy

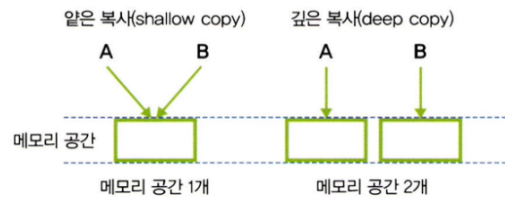
original = [[1, 2], 3]
copy_o = copy.deepcopy(original) ----- original 값을 copy_o에 깊은 복사(copy.deepcopy())
print(copy_o)
copy_o[0] = 100 ----- copy_o의 [1, 2] 값을 100으로 변경
print(copy_o)
print(original)

append = copy.deepcopy(original)
append[0].append(4) ----- 첫 번째 리스트([1, 2])에 4를 추가
print(append)
print(original)
```

결과

```
[[1, 2], 3]
[100, 3]
[[1, 2], 3]
[[1, 2, 4], 3]
[[1, 2], 3]
```

- 얇은 복사와 깊은 복사의 차이



```
# VGG 모델 정의
class VGG(nn.Module):
    def __init__(self, features, output_dim):
        super().__init__()
        self.features = features # VGG 모델에 대한 매개변수에서 받아 온 feature
        s 값을 self.features에 넣어 줌
        self.avgpool = nn.AdaptiveAvgPool2d(7)
        self.classifier = nn.Sequential(
            nn.Linear(512*7*7, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(4096, output_dim)
        ) # 완전연결층과 출력층 정의

    def forward(self, x):
        x = self.features(x)
        x = self.avgpool(x)
        h = x.view(x.shape[0], -1)
        x = self.classifier(h)
        return x, h
```

```
# 모델 유형 정의
vgg11_config = [64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512,
'M'] # 8(합성곱층) + 3(풀링층) = 11(전체계층) = VGG11
```

코드 6-41 모델 유형 정의

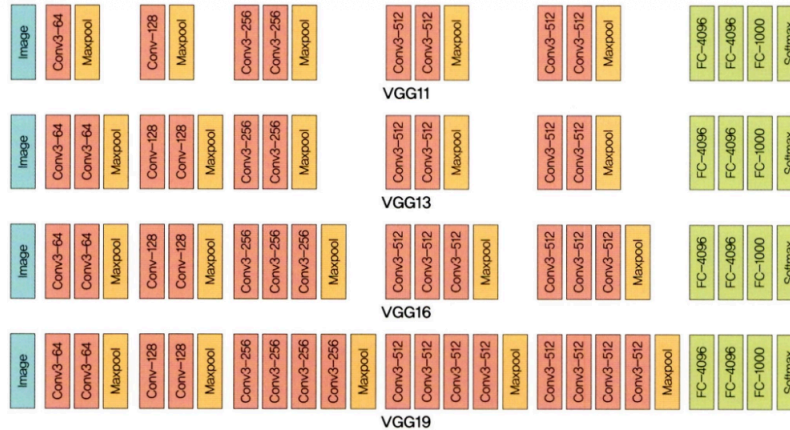
```
vgg11_config = [64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M'] .....
8(합성곱층)+3(풀링층)=11(전체 계층)=VGG11

vgg13_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512,
'M'] .....
10(합성곱층)+3(풀링층)=13(전체 계층)=VGG13

vgg16_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 'M', 512, 512, 512, 'M',
512, 512, 512, 'M'] .....
13(합성곱층)+3(풀링층)=16(전체 계층)=VGG16

vgg19_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 256, 'M', 512, 512, 512,
512, 'M', 512, 512, 512, 512, 'M'] .....
16(합성곱층)+3(풀링층)=19(전체 계층)=VGG19
```

▼ 그림 6-15 VGG의 다양한 모델에 대한 네트워크



```
# VGG 계층 정의
def get_vgg_layers(config, batch_norm):
    layers=[]
    in_channels=3

    for c in config: # vgg11_config 값들을 가져옴
        assert c == 'M' or isinstance(c, int) # 설명
        if c == 'M': # 불러온 값이 'M'이면 MaxPool2d 적용
            layers += [nn.MaxPool2d(kernel_size=2)]
        else: # 불러온 값이 숫자이면 Conv2d 적용
            conv2d = nn.Conv2d(in_channels, c, kernel_size=3, padding=1)
            if batch_norm: # batch normalization 적용할지에 대한 코드
                layers += [conv2d, nn.BatchNorm2d(c), nn.ReLU(inplace=True)] # 배치 정규화가 적용될 경우 배치 정규화+ReLU 적용
            else:
                layers += [conv2d, nn.ReLU(inplace=True)] # 배치 정규화가 적용되지 않을 경우 ReLU만 적용
            in_channels=c

    return nn.Sequential(*layers) # 네트워크의 모든 계층 반환
```

- 설명
조건문을 정의

```
assert c == 'M' or isinstance(c, int)
```

- `assert c == 'M'`

`assert`는 뒤의 조건이 `true`가 아니면 에러 발생. 따라서 `c == 'M'`이 아니면 에러 발생

```
a = 1
assert a == 11
```

조건이 `True`가 아니기 때문에 다음과 같은 오류가 발생합니다.

```
-----
AssertionError                                Traceback (most recent call last)
<ipython-input-1-b211624433f8> in <module>
      1 a = 1
----> 2 assert a == 11

AssertionError:
```

- `isinstance(c, int)`

`isinstance`는 주어진 조건이 `True`인지 판단.

```
print(isinstance(1, int)) ----- 1이 integer인지 판단
print(isinstance(1.2, int)) ----- 1.2가 integer인지 판단
print(isinstance('deep learning', str)) ----- deep learning이 string인지 판단
```

다음은 `isinstance`로 조건이 `True/False`인지 확인한 결과입니다.

```
True
False
True
```

`assert c == 'M' or isinstance(c, int)` : `c`가 'M'이 아니거나 `int`가 아니라면 오류 발생

```
# 모델 계층 생성
```

```
vgg11_layers = get_vgg_layers(vgg11_config, batch_norm=True)
```

- `batch_norm(Batch Normalization)` 은 데이터의 평균을 0으로, 표준편차를 1로 분포시키는 것. 각 계층에서 입력 데이터의 분포는 앞 계층에서 업데이트된 가중치에 따라 변함. 즉, 각 계층마다 변화되는 분포는 학습 속도를 늦출 뿐 아니라 학습도 어렵게 함. 따라서 각 계층의 입력에 대한 분산을 평균 0, 표준편차 1로 분포시키는 것이 `batch_norm`

```
# VGG11 계층 확인
```

```
print(vgg11_layers)
```

▼ 코드 실행 결과

```

Sequential(
  (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (2): ReLU(inplace=True)
  (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (4): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (5): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (6): ReLU(inplace=True)
  (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (8): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (9): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (10): ReLU(inplace=True)
  (11): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (12): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (13): ReLU(inplace=True)
  (14): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (15): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (16): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (17): ReLU(inplace=True)
  (18): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (19): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (20): ReLU(inplace=True)
  (21): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (22): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (23): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  ...
  (26): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (27): ReLU(inplace=True)
  (28): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
)

```

```

# VGG11 전체에 대한 네트워크
OUTPUT_DIM = 2 # 개와 고양이 두 개의 클래스 사용
model = VGG(vgg11_layers, OUTPUT_DIM)
print(model)

```

▼ 코드 실행 결과

```

VGG(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (4): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (5): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): ReLU(inplace=True)
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (8): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (10): ReLU(inplace=True)
    (11): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (12): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (13): ReLU(inplace=True)
    (14): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (15): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (16): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (17): ReLU(inplace=True)
    (18): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (19): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (20): ReLU(inplace=True)
    (21): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (22): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    ...
    (5): Dropout(p=0.5, inplace=False)
    (6): Linear(in_features=4096, out_features=2, bias=True)
  )
)

```

```
# VGG11 사전 훈련된 모델 사용
import torchvision.models as models
pretrained_model = models.vgg11_bn(pretrained=True) # 설명
print(pretrained_model)
```

▼ 코드 실행 결과

```
/opt/homebrew/Caskroom/miniforge/base/envs/dl-euron/lib/python3.10/site-packages/torchvision/models/
warnings.warn(
/opt/homebrew/Caskroom/miniforge/base/envs/dl-euron/lib/python3.10/site-packages/torchvision/models/
warnings.warn(msg)
Downloading: "https://download.pytorch.org/models/vgg11_bn-6002323d.pth" to /Users/eunseoshin/.cache
100%|██████████| 507M/507M [04:00<00:00, 2.21MB/s]
VGG(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (4): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (5): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): ReLU(inplace=True)
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (8): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (10): ReLU(inplace=True)
    (11): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (12): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (13): ReLU(inplace=True)
    (14): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (15): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (16): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (17): ReLU(inplace=True)
    (18): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (19): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (20): ReLU(inplace=True)
    (21): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (22): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    ...
    (5): Dropout(p=0.5, inplace=False)
    (6): Linear(in_features=4096, out_features=1000, bias=True)
  )
)
```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

- 배치 정규화가 적용된 사전 훈련된 VGG11 모델을 사용하기 위해서는 다음과 같은 파라미터를 사용한다.

```
pretrained_model = models.vgg11_bn(pretrained=True)
```

- `models.vgg11_bn` 은 VGG11 기본 모델에 배치 정규화가 적용된 모델을 사용하겠다는 의미
- `pretrained=True` 로 설정하면 사전 훈련된 모델을 사용(미리 학습된 파라미터 값들을 사용)하겠다는 의미

```
# 이미지 데이터 전처리
train_transforms = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomRotation(5),
    transforms.RandomHorizontalFlip(0.5),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.
225])
```

```
] # 설명
```

```
test_transforms = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.
225])
])
```

- 설명

```
transforms.Compose([transforms.Resize(256, 256)), transforms.RandomRot
ation(5)])
```

- `transforms.Resize` : 이미지를 주어진 크기로 재조정
- `transforms.RandomRotation` : 5도 이하로 이미지 회전

```
# ImageFolder를 이용해 데이터셋 불러오기
train_path = '/Users/eunseoshin/DL-pytorch/chap06/data/catanddog/train'
test_path = '/Users/eunseoshin/DL-pytorch/chap06/data/catanddog/test'

train_dataset = torchvision.datasets.ImageFolder(
    train_path,
    transform=train_transforms
)

test_dataset = torchvision.datasets.ImageFolder(
    test_path,
    transform=test_transforms
)

print(len(train_dataset)), print(test_dataset)
```

▼ 코드 실행 결과

```
529
Dataset ImageFolder
  Number of datapoints: 12
  Root location: /Users/eunseoshin/DL-pytorch/chap06/data/catanddog/test
  StandardTransform
  Transform: Compose(
    Resize(size=(256, 256), interpolation=bilinear, max_size=None, antialias=True)
    ToTensor()
    Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
  )
(None, None)
```

```
# 훈련과 검증 데이터 분할
VALID_RATIO = 0.9
n_train_examples = int(len(train_dataset) * VALID_RATIO) # 전체 훈련 데이터
중 90%를 훈련 데이터셋으로 사용
n_valid_examples = len(train_dataset) - n_train_examples # 전체 훈련 데이터
중 10%를 검증 데이터셋으로 사용

train_data, valid_data = data.random_split(train_dataset,

[n_train_examples, n_valid_examples]) # 설명
```

- 설명

파이썬의 `random_split()` 은 훈련과 검증 데이터셋을 나누는 용도로 사용하며 다음과 같은 파라미터를 사용한 다. (데이터로더로 데이터가 넘어간 이후에는 분리가 불가능하므로 데이터셋 단계에서 진행해야 함)

```
train_data, valid_data = data.random_split(train_dataset,

[n_train_examples, n_valid_examples])
```

- 첫 번째 파라미터 : 분할에 사용될 데이터셋
- 두 번째 파라미터 : 훈련과 검증 데이터셋의 크기 지정. [훈련 데이터셋 크기, 검증 데이터셋 크기]

```
# 검증 데이터 전처리
valid_data = copy.deepcopy(valid_data)
valid_data.dataset.transform = test_transforms

# 훈련, 검증, 테스트 데이터셋 수 확인
print(f'Number of training examples: {len(train_data)}')
print(f'Number of validation examples: {len(valid_data)}')
print(f'Number of testing examples: {len(test_dataset)}')
```

▼ 코드 실행 결과

```
Number of training examples: 476
Number of validation examples: 53
Number of testing examples: 12
```

```
# 메모리로 데이터 불러오기
BATCH_SIZE = 128
train_iterator = data.DataLoader(train_data, shuffle=True, batch_size=BATCH_SIZE)
valid_iterator = data.DataLoader(valid_data, batch_size=BATCH_SIZE)
test_iterator = data.DataLoader(test_dataset, batch_size=BATCH_SIZE)
```

```
# 옵티마이저, 손실함수 정의
optimizer = optim.Adam(model.parameters(), lr=1e-7)
criterion = nn.CrossEntropyLoss()

model = model.to(device)
criterion = criterion.to(device)
```

```
# 모델 정확도 측정 함수
def calculate_accuracy(y_pred, y):
    top_pred = y_pred.argmax(1, keepdim=True)
    correct = top_pred.eq(y.view_as(top_pred)).sum() # 설명
    acc = correct.float() / y.shape[0]
    return acc
```

- 설명

예측과 정답이 일치하는 경우 그 개수의 합을 correct 변수에 저장

```
correct = top_pred.eq(y.view_as(top_pred)).sum()
```

- `top_pred.eq` : 서로 같은지 비교
- `view_as(other)` : other의 텐서 크기를 사용하겠다는 의미
- `sum()` : 합계를 구하는 함수

```
# 모델 학습 함수 정의
def train(model, iterator, optimizer, criterion, device):
    epoch_loss = 0
    epoch_acc = 0

    model.train()

    for (x, y) in iterator:
        x = x.to(device)
        y = y.to(device)

        optimizer.zero_grad()
        y_pred, _ = model(x)
        loss = criterion(y_pred, y)
        acc = calculate_accuracy(y_pred, y)
        loss.backward()
        optimizer.step()

        epoch_loss += loss.item()
        epoch_acc += acc.item()

    return epoch_loss / len(iterator), epoch_acc / len(iterator)
```

```
# 모델 성능 측정 함수
def evaluate(model, iterator, criterion, device):
    epoch_loss = 0
    epoch_acc = 0

    model.eval()
    with torch.no_grad():
        for (x, y) in iterator:
            x = x.to(device)
            y = y.to(device)
            y_pred, _ = model(x)
            loss = criterion(y_pred, y)
            acc = calculate_accuracy(y_pred, y)
            epoch_loss += loss.item()
            epoch_acc += acc.item()
    return epoch_loss / len(iterator), epoch_acc / len(iterator)
```

```
# 학습 시간 측정 함수
def epoch_time(start_time, end_time):
    elapsed_time = end_time - start_time
    elapsed_mins = int(elapsed_time / 60)
    elapsed_secs = int(elapsed_time - (elapsed_mins * 60))
    return elapsed_mins, elapsed_secs
```

```
# 모델 학습
EPOCHS = 5
best_valid_loss = float('inf')
for epoch in range(EPOCHS):
    start_time = time.monotonic()
    train_loss, train_acc = train(model, train_iterator, optimizer, criterion, device)
    valid_loss, valid_acc = evaluate(model, valid_iterator, criterion, device)

    if valid_loss < best_valid_loss:
        best_valid_loss = valid_loss
        torch.save(model.state_dict(), '../chap06/data/VGG-model.pt')

    end_time = time.monotonic()
    epoch_mins, epoch_secs = epoch_time(start_time, end_time)

    print(f'Epoch: {epoch+1:02} | Epoch Time: {epoch_mins}m {epoch_secs}s')
    print(f'\tTrain Loss: {train_loss:.3f} | Train Acc: {train_acc*100:.2f}%')
```

```
print(f'\t Valid. Loss: {valid_loss:.3f} | Valid. Acc: {valid_acc*100:.2f}%')
```

▼ 코드 실행 결과

```
Epoch: 01 | Epoch Time: 2m 34s
      Train Loss: 0.702 | Train Acc: 51.25%
      Valid. Loss: 0.693 | Valid. Acc: 52.83%
Epoch: 02 | Epoch Time: 2m 34s
      Train Loss: 0.699 | Train Acc: 50.90%
      Valid. Loss: 0.693 | Valid. Acc: 54.72%
Epoch: 03 | Epoch Time: 2m 44s
      Train Loss: 0.699 | Train Acc: 52.18%
      Valid. Loss: 0.693 | Valid. Acc: 43.40%
Epoch: 04 | Epoch Time: 2m 51s
      Train Loss: 0.695 | Train Acc: 53.45%
      Valid. Loss: 0.694 | Valid. Acc: 47.17%
Epoch: 05 | Epoch Time: 3m 1s
      Train Loss: 0.699 | Train Acc: 49.58%
      Valid. Loss: 0.693 | Valid. Acc: 50.94%
```

```
# 테스트 데이터셋을 이용한 모델 성능 측정
model.load_state_dict(torch.load('../chap06/data/VGG-model.pt'))
test_loss, test_acc = evaluate(model, test_iterator, criterion, device)
print(f'Test Loss: {test_loss:.3f} | Test Acc: {test_acc*100:.2f}%')
```

▼ 코드 실행 결과

```
/var/folders/4g/3301trc50fz6\_vl0qx1b71r00000gn/T/ipykernel\_13006/242
model.load_state_dict(torch.load('../chap06/data/VGG-model.pt'))
Test Loss: 0.693 | Test Acc: 50.00%
```

```
# 테스트 데이터셋을 이용한 모델의 예측 확인 함수
def get_predictions(model, iterator):
    model.eval()
    images = []
    labels = []
    probs = []

    with torch.no_grad():
        for (x, y) in iterator:
            x = x.to(device)
            y_pred, _ = model(x)
            y_prob = F.softmax(y_pred, dim = -1)
            top_pred = y_prob.argmax(1, keepdim = True) # 설명(1)
            images.append(x.cpu())
            labels.append(y.cpu())
            probs.append(y_prob.cpu())
```

```
images = torch.cat(images, dim = 0) # 설명(2)
labels = torch.cat(labels, dim = 0)
probs = torch.cat(probs, dim = 0)
return images, labels, probs
```

- 설명 (1)

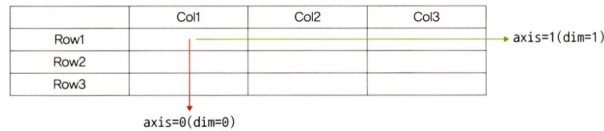
argmax는 배열에서 가장 큰 값의 인덱스를 찾을 때 사용

```
top_pred = y_prob.argmax(1, keepdim = True)
```

- 첫 번째 파라미터 : 행(axis=0) 또는 열(axis=1)을 따라 가장 큰 값의 색인을 찾음.
- `keepdim` : True의 경우 출력 텐서를 입력과 동일한 크기로 유지하겠다는 의미

- 설명 (2)

`torch.cat` 은 텐서를 연결할 때 사용.



차원 설명

```
import torch
x = torch.Tensor([[1, 2, 3], [2, 3, 4]])
y = torch.Tensor([[4, 5, 6], [5, 6, 7]])

print(torch.cat([x], dim=0)) ----- 행을 기준(dim=0)으로 x를 이어 붙입니다.
print('-----')
print(torch.cat([x, y])) ----- 단순히 x와 y를 결합하라는 의미이며 출력 결과는 (4x3) 형태가 됩니다.
print('-----')
print(torch.cat([x, y], dim=0)) ----- 행을 기준(dim=0)으로 x와 y를 이어 붙이면 (4x3) 텐서가 됩니다.
print('-----')
print(torch.cat([x, y], dim=1)) ----- 열을 기준(dim=1)으로 x와 y를 이어 붙이면 (2x6) 텐서가 됩니다.
```

결과

```
tensor([[1., 2., 3.],
        [2., 3., 4.]])
-----
tensor([[1., 2., 3.],
        [2., 3., 4.]])
```

```

[4., 5., 6.],
[5., 6., 7.]]
-----
tensor([[1., 2., 3.],
        [2., 3., 4.],
        [4., 5., 6.],
        [5., 6., 7.]])
-----
tensor([[1., 2., 3., 4., 5., 6.],
        [2., 3., 4., 5., 6., 7.]])

```

```

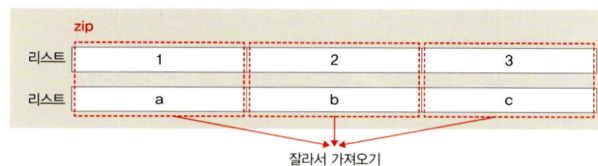
# 예측 중 정확하게 예측한 것을 추출
images, labels, probs = get_predictions(model, test_iterator)
pred_labels = torch.argmax(probs, 1) # 설명(1)
corrects = torch.eq(labels, pred_labels)
correct_examples = []

for image, label, prob, correct in zip(images, labels, probs, corrects): #
설명(2)
    if correct:
        correct_examples.append((image, label, prob))

correct_examples.sort(reverse = True, key = lambda x: torch.max(x[2], dim
= 0).values) # 설명(3)

```

- 설명 (1)
argmax는 최댓값을 갖는 인덱스 반환
- 설명 (2)



zip()은 여러 개의 리스트(혹은 튜플)를 합쳐서 새로운 튜플 타입으로 반환

```

a = [1, 2, 3]
b = ['a', 'b', 'c']

for x, y in zip(a, b): ----- a, b가 리스트로 구성되었기 때문에 for 문을 통해 반복하여 값을 가져옵니다.
    print(x, y)

```

다음은 zip을 적용한 결과입니다.

```

1 a
2 b
3 c

```

- 설명 (3)
데이터를 정렬하기 위해 sort() 메서드 이용. 파라미터는 다음과 같다.

```
correct_examples.sort(reverse = True, key = lambda x: torch.max(x[2], dim = 0).values)
```

- `reverse` : 내림차순으로 정렬

```
x = [1, 7, 3, 9] ----- 정렬되어 있지 않은 x라는 텐서 생성
x.sort(reverse=True) ----- x에 내림차순 적용
print('x:', x)
```

다음과 같이 x가 내림차순으로 정렬되었습니다.

```
x: [9, 7, 3, 1]
```

- `key` : 데이터를 정렬할 때 key 값을 가지고 정렬하며, 기본값은 오름차순.

```
def 함수명(매개변수):          lamda 매개변수 : 반환될 결과값
    return 반환될 결과값
```

람다 함수는 정의와 동시에 사용할 수 있지만 함수명이 없고, 저장된 변수가 없기 때문에 재사용 불가능

```
# 이미지 출력을 위한 전처리
def normalize_image(image):
    image_min = image.min()
    image_max = image.max()
    image.clamp_(min = image_min, max = image_max) # torch.clamp는 주어진 최소, 최대의 범주에 이미지가 위치하도록 함
    image.add_(-image_min).div_(image_max - image_min + 1e-5) # 설명
    return image
```

• 설명

`torch.add` 는 말 그대로 더하라는 메서드.

- `torch.add`와 `torch.add_`의 차이

```
x = torch.tensor([1, 2])
y = x.add(10) ----- torch.add 적용
print(y)
print(x is y)
print('-----')
y = x.add_(10) ----- torch.add_ 적용
print(y)
print(x is y)
```

다음과 같이 `torch.add`와 `torch.add_`의 결과는 같지만, x가 y와 같은지(x is y) 물었을 때는 다른 값을 출력합니다.

```
tensor([11, 12])
False
-----
tensor([11, 12])
True
```

```

# 모델이 정확하게 예측한 이미지 출력 함수
def plot_most_correct(correct, classes, n_images, normalize = True):
    rows = int(np.sqrt(n_images)) # np.sqrt는 제곱근을 계산
    cols = int(np.sqrt(n_images))
    fig = plt.figure(figsize = (25, 20))
    for i in range(rows*cols):
        ax = fig.add_subplot(rows, cols, i+1) # 출력하려는 그래프 개수만큼 subplot 만들기
        image, true_label, probs = correct[i]
        image = image.permute(1, 2, 0) # 설명
        true_prob = probs[true_label]
        correct_prob, correct_label = torch.max(probs, dim = 0)
        true_class = classes[true_label]
        correct_class = classes[correct_label]

        if normalize: # 본래 이미지대로 출력하기 위해 normalize_image 함수 호출
            image = normalize_image(image)

        ax.imshow(image.cpu().numpy())
        ax.set_title(f'true label: {true_class} ({true_prob:.3f})\n' \
                    f'pred label: {correct_class} ({correct_prob:.3f})')
        ax.axis('off')

    fig.subplots_adjust(hspace = 0.4)

```

- 설명

`image.permute` 는 축을 변경할 때 사용

```

x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print(x)
x.permute(1, 0) ----- 차원(축) 0과 1을 바꿉니다.

```

다음은 0과 1의 차원(축)이 변경된 결과입니다. 즉, 행과 열이 변경되었습니다.

```

tensor([[1, 2, 3],
        [4, 5, 6]])
tensor([[1, 4],
        [2, 5],
        [3, 6]])

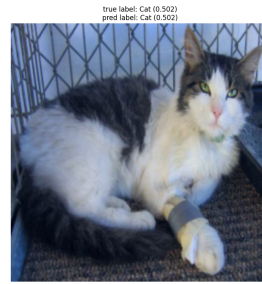
```

```

# 예측 결과 이미지 출력
classes = test_dataset.classes
N_IMAGES = 5
plot_most_correct(correct_examples, classes, N_IMAGES)

```

- ▼ 코드 실행 결과

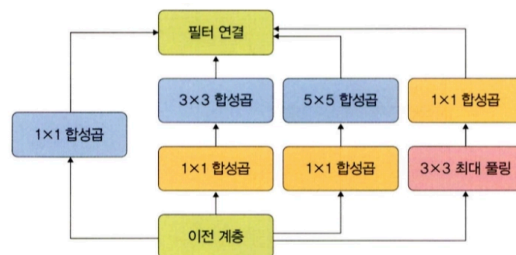


6.1.4 GoogLeNet

GoogLeNet은 주어진 하드웨어 자원을 최대한 효율적으로 이용하면서 학습 능력은 극대화할 수 있는 깊고 넓은 신경망.

깊고 넓은 신경망을 위해 인셉션(inception) 모듈을 추가함. 인셉션 모듈에서는 특징을 효율적으로 추출하기 위해 1×1 , 3×3 , 5×5 의 합성곱 연산을 각각 수행함. 3×3 최대 풀링은 입력과 출력의 높이와 너비가 같아야 하므로 풀링 연산에서는 드물게 패딩을 추가해야 함. 결과적으로 GoogLeNet에 적용된 해결 방법은 희소 연결(sparse connectivity).

CNN은 합성곱, 풀링, 완전연결층이 서로 밀집(dense)하게 연결되어 있음. 뿔뿔하게 연결된 신경망 대신 관련성(correlation)이 높은 노드끼리만 연결하는 방법을 희소 연결이라고 함. 이것으로 연산량이 적어지며 과적합도 해결 가능.



인셉션 모듈의 네 가지 연산은 다음과 같다.

- 1×1 합성곱
- 1×1 합성곱 + 3×3 합성곱
- 1×1 합성곱 + 5×5 합성곱
- 3×3 최대 풀링(maxpooling) + 1×1 합성곱(convolutional)

딤러닝을 이용해 ImageNet과 같은 대회에 참여하거나 서비스를 제공하려면 대용량 데이터를 학습해야 함. 심층 신경망의 아키텍처에서 계층이 넓고(뉴런이 많고) 깊으면(계층이 많으면) 인식률은 좋아지지만, 과적합이나 기울기

소멸 문제를 비롯한 학습 시간 지연과 연산 속도 등의 문제가 있음. 특히 합성곱 신경망에서 이러한 문제들이 자주 나타나는데, GoogLeNet으로 이러한 문제를 해결할 수 있음.